

Technical Analysis of a Quad-Socket Supermicro X9QRi-F+ Storage Platform

1. Motherboard Overview: C602 Chipset and Quad-CPU Architecture

The Supermicro X9QRi-F+ is a quad-socket server motherboard built on the Intel C602 chipset, supporting four LGA 2011 ("Socket R") CPU sockets ¹. It is designed for Intel Xeon E5-4600 series processors (both the original **v1** and Ivy Bridge-EP **v2** generations) with up to 12 cores per CPU ¹. The C602 chipset provides traditional server platform features such as multiple SATA ports, PCI Express lane management, and enterprise reliability features (Intel RSTe RAID, AMT/ME manageability, etc.). In a four-CPU configuration, the processors communicate via Intel's **QuickPath Interconnect (QPI)** links, forming a coherent Non-Uniform Memory Access (NUMA) system. Each Xeon E5-4600 v2 CPU has two QPI links (8.0 GT/s each) to connect with its neighbors ² ¹. The board's architecture uses a glueless quad-CPU interconnect: the CPUs are arranged such that each CPU is directly connected to two others via QPI, creating a mesh/ring topology (one intermediate hop to reach the fourth node). This ensures reasonably low-latency communication between processors for memory access and cache coherency, albeit with some NUMA effects when accessing remote memory (discussed in Section 3). The Intel C602 PCH (Platform Controller Hub) itself interfaces with one of the CPUs (typically CPU1) via a DMI 2.0 link (essentially a PCIe x4 connection) to handle peripheral I/O ³. This means that onboard devices (SATA ports, etc.) are "owned" by a specific CPU; in fact, the X9QRi-F+ manual notes that certain features (like the onboard NICs) won't function unless the CPU socket linked to them is populated ⁴. In summary, the X9QRi-F+ provides a robust quad-CPU infrastructure: **Intel C602 chipset**, four LGA2011 sockets, and a high-speed QPI network linking the CPUs for cache-coherent memory sharing ¹.

Supported CPUs: The board supports the entire Intel Xeon E5-4600 family for 4-socket servers, including both v1 (Sandy Bridge-EP, up to 8 cores) and v2 (Ivy Bridge-EP, up to 12 cores) models ¹. A BIOS update (v3.0 or later) is required to use the Ivy Bridge-EP v2 processors (launched in 2014) ⁵ ⁶. With an appropriate BIOS, one can install top-end chips like the Xeon E5-4657L v2 (12 cores @2.4 GHz) or E5-4650 v2 (10 cores @2.4 GHz), which were among the highest-performance options for this platform ⁷. The board is **proprietary EATX** in form factor (approximately 16.4" × 16.8"), meaning it's larger than standard EATX and requires a roomy chassis ⁸ ⁹. Supermicro offered compatible chassis (e.g. 4U models with high-wattage PSUs, see Section 7) to accommodate this large board and its four CPUs. Key integrated components on the motherboard include an on-board Baseboard Management Controller (BMC) for IPMI 2.0 remote management and a basic VGA controller (Nuvoton WPCM450 with Matrox G200 core) for console video ¹⁰ ¹¹. The board's **dual Intel i350 Gigabit NICs** and IPMI module are also noteworthy and will be detailed in Section 6.

Overall, the X9QRi-F+ provides the foundation for a high-end **quad-CPU server**, with a layout and feature set optimized for heavy multi-threaded workloads and large memory capacity. Its chipset and architecture allow the four CPUs to function as a cohesive system suitable for enterprise tasks. In the context of a storage hub, the board's ability to host many cores, large RAM, and multiple high-bandwidth I/O cards (PCIe slots) is a major asset – albeit one must design around the NUMA nature and I/O attachment specifics (for

example, ensuring that all four CPU sockets are populated to enable full functionality of PCIe slots and onboard devices) ⁴ .

2. Processor Specifications: Intel Xeon E5-4600 v2 (E5-4650 v2 / E5-4657L v2)

The X9QRi-F+ can house up to four Xeon E5-4600 v2 processors, which are **Ivy Bridge-EP** architecture CPUs built on 22 nm process (launched Q1'2014) ¹² ¹³ . For a high-performance configuration, we consider the top models in this family: for example, the **Intel Xeon E5-4650 v2** (10 cores, 20 threads) and the **Xeon E5-4657L v2** (12 cores, 24 threads). Each CPU runs at a base frequency of 2.40 GHz, with turbo boost up to ~2.9 GHz on single cores ¹⁴ ¹⁵ . These chips feature large last-level caches (**25 MB L3** on the 10-core E5-4650 v2, and **30 MB L3** on the 12-core E5-4657L v2) to accelerate memory access across their cores ¹⁶ ¹⁷ . Each core also has a dedicated 64 KB L1 (32 KB instruction + 32 KB data) and 256 KB L2 cache, totaling **640 KB L1** and **2.5 MB L2** per processor for the 10-core model ¹⁷ . The “L” variant (4657L v2) offers more cores but is optimized for lower voltage; it carries a slightly higher TDP of 115 W, whereas the 4650 v2 is rated at **95 W TDP** ¹⁸ ¹⁹ . In a fully loaded system with four CPUs, the aggregate thermal design power is ~380–460 W just for the processors, which has implications for power and cooling (see Section 7).

Microarchitecture and Features: As Ivy Bridge-EP chips, the E5-4600 v2 family brings ISA enhancements and server-oriented features: they support **Intel 64 (x86-64)**, Hyper-Threading (2 threads per core), and Turbo Boost 2.0 ²⁰ ²¹ . They also include Intel AVX (256-bit vector instructions) and AES-NI acceleration for cryptography ²² . Notably, these CPUs are designed exclusively for 4-socket operation (“**Scalability: 4S Only**”) ²³ – they have the necessary QPI links (2 links each) to connect into a quad-processor topology. The QuickPath Interconnect runs at **8.0 GT/s** on these models, providing high bandwidth between CPUs for memory access and cache coherence ⁷ ²⁴ . In terms of memory interface, each E5-4600 v2 has a four-channel DDR3 memory controller (integrated on the CPU), supporting speeds up to DDR3-1866 (in ideal configurations) ²⁵ ²⁶ . At maximum rated speed and with four channels populated, a single CPU can achieve roughly **60 GB/s of memory bandwidth** ²⁷ . However, note that maximum memory speed might be reduced in multi-DIMM configurations (e.g., with two DIMMs per channel the speed typically falls to DDR3-1600 or 1333 – see Section 3).

NUMA and Multi-Processor Implications: With four E5-4600 v2 CPUs, the system is divided into **4 NUMA nodes** (each CPU and its local memory is one node). Memory access is fastest when a core accesses the memory attached to its own socket, taking advantage of the ~60 GB/s local bandwidth ²⁷ . Accessing memory attached to another CPU involves traversing one or two QPI links (with each QPI link providing up to ~8 GT/s, roughly 32 GB/s half-duplex). This adds latency and slightly reduces bandwidth available to remote memory. In practice, this means a Linux OS will identify 4 NUMA nodes and can schedule workloads accordingly. For optimal performance, memory-intensive processes should be “NUMA-aware” – e.g. Linux can use NUMA balancing or one can pin certain tasks to CPUs and allocate memory on the same node. The **cache coherence** between the processors is maintained via a MESIF protocol over QPI; the large L3 caches on each CPU help reduce the need for remote memory fetches by caching shared data. Still, workloads that heavily involve cross-CPU communication may see higher latency due to inter-socket links. For a storage server, this NUMA effect might surface if (for instance) an I/O device on one CPU is being used by a process running on a different CPU, causing data to hop over QPI. Administrators can mitigate this by aligning interrupt handling and processes to the CPU that “owns” the storage controller, or by enabling NUMA I/O optimizations in the OS.

Power and Thermal Profile: The TDP per chip (95–115 W as noted) implies that four CPUs can consume ~400 W at full load. These Xeons use Intel's power management features (SpeedStep, idle states, power throttling) to reduce consumption when idle ²⁸, but under load they will draw significant power and generate substantial heat. Each CPU's maximum case temperature (T_{case}) is around 75°C ²⁹, so adequate cooling is required to maintain safe operation – typically active heatsinks with high airflow in a server chassis (Section 7 covers cooling considerations). In exchange for this power usage, the four E5-4650 v2 processors collectively offer **40 physical cores / 80 threads** (or up to 48 cores/96 threads with the 12-core models) and enormous compute throughput for parallel tasks. For perspective, a single E5-4650 v2 delivers around 2.4 GHz × 10 cores × 8 FLOPs per cycle (with AVX) in peak theoretical performance – on the order of ~192 GFLOPS for double precision, per CPU. Four such CPUs could exceed 0.75 TFLOPS of FP64 theoretical throughput. While raw FLOPS aren't typically the limiting factor for a storage hub, this illustrates that the platform has ample CPU muscle for auxiliary tasks like compression, encryption (with AES-NI), deduplication, or serving many clients concurrently.

In summary, the **Xeon E5-4600 v2** processors used in the X9QRi-F+ bring high core counts, large caches, and advanced server features (VT-x/VT-d virtualization, AES-NI security, etc. ²²) to support enterprise workloads. Their multi-socket capability and memory bandwidth make them well-suited to a high-performance storage server that might handle not just disk I/O, but also processing of data (e.g. RAID parity calculations, ZFS checksums, or application workloads running on the same server). The trade-off is the high power consumption and heat, as well as being a generation behind modern CPUs in per-core performance. We will revisit how these CPUs compare to newer platforms in Section 10.

3. Memory Configuration: 82 GB DDR3 ECC across Four NUMA Nodes

Memory Architecture: The X9QRi-F+ has an impressive **32 DIMM slots** spread across the four CPU sockets ³⁰. Each CPU socket connects to 8 DIMM slots (since each Xeon E5-4600 has a 4-channel memory controller and the board supports 2 DIMMs per channel). This yields a theoretical maximum of **1 TB DDR3** memory using 32 GB LR-DIMMs in all slots ³¹ ³². In practice, our configuration has a total of **82 GB DDR3 ECC** RAM installed. This unusual capacity suggests a non-symmetrical or mixed-DIMM setup – for example, one might have a mix of 8 GB and 4 GB modules, or perhaps one socket populated with less memory than the others. Ideally, for balanced performance, memory should be evenly distributed across the four CPUs. A typical balanced configuration would populate the same number of DIMMs on each CPU (ensuring each CPU has equal local memory). An uneven total like 82 GB indicates one or more CPUs have an extra module (or a higher-capacity module) compared to the others. While the system will still function, such imbalance can lead to NUMA asymmetry (one node has more memory) and potentially some performance variations if that memory is heavily used. Generally, one would strive to populate either 80 GB (e.g. 4 × 8 GB on each CPU = 32 GB per CPU, total 128 GB) or 84 GB (e.g. mixing to get 21 GB per CPU) rather than 82; however, assuming 82 GB is what's available, the memory controller will interleave memory access in each CPU's channels for those installed DIMMs.

DIMM Layout and ECC: Each memory slot on the X9QRi-F+ accepts DDR3 registered ECC memory (RDIMM), and the board also supports load-reduced DIMMs (LRDIMM) and even unbuffered ECC (UDIMM) in smaller sizes ³³ ²⁶. Supported module sizes range from 1 GB up to 32 GB ²⁶. In enterprise servers, **ECC (Error-Correcting Code)** memory is crucial for reliability: the memory system can detect and correct single-bit errors and detect (but not correct) double-bit errors ³⁴. This significantly reduces the risk of memory corruption causing data integrity issues or crashes, which is important for a storage hub handling critical data. The installed 82 GB is ECC-enabled, meaning any single-bit memory error will be transparently fixed

and logged (using an extra parity bit lane in the 72-bit DIMMs). The board supports both standard 1.5 V DDR3 and low-voltage 1.35 V DDR3 DIMMs ³⁵, so one could use energy-efficient memory if available (many 16 GB DIMMs are 1.35 V, for instance).

Memory Speed and Bandwidth: The official memory speed support goes up to **DDR3-1866** MT/s on this board (with Ivy Bridge v2 CPUs) ³¹. However, achieving 1866 MT/s typically requires a limited configuration (such as only one DIMM per channel). According to Supermicro's documentation and industry practice, if two DIMMs per channel are populated, the maximum speed often drops to **1600 MT/s** ³⁶. In fact, an STH forum note for this board indicates that *"you can't run at 1866 MHz with multiple DIMMs per channel... memory will be clocked at no more than 1600 MHz"* ³⁶. Therefore, with 32 slots likely having at least 1 DIMM per channel (and possibly 2 in some channels), the realistic memory clock is 1600 MHz. At DDR3-1600, each channel provides ~12.8 GB/s of bandwidth, so each CPU maxes out around **51.2 GB/s** of local memory bandwidth (which aligns with Intel's numbers for 1600 MT/s operation) ²⁷. If the configuration had been 1866, each CPU could reach ~59.7 GB/s ²⁷, but again that's for a light DIMM load. The total system memory bandwidth (all four CPUs combined) could be on the order of 200 GB/s in theory, but in practice memory traffic does not perfectly scale that way due to NUMA locality and the fact that each CPU mostly handles its own memory accesses.

In our 82 GB setup, it's likely that some channels are not fully populated. For example, one possible distribution might be: CPU1 has 20 GB (two 8 GB + one 4 GB), CPU2 has 20 GB, CPU3 has 20 GB, CPU4 has 22 GB. This would total 82 GB and give roughly balanced 2-channel vs 2-channel population. In such a case, not all four channels per CPU are used; using fewer channels can actually allow higher speed (if one channel per CPU was empty, the others might run at 1866). However, losing a channel reduces peak bandwidth on that CPU. It's a trade-off: ideally one populates either 4 or 8 DIMMs per CPU for symmetry. Given that 82 is a non-power-of-two number, we suspect a slightly non-optimal population. Nonetheless, **82 GB is plenty of RAM** for a storage server, allowing for large disk caches and buffers. Linux can utilize the excess memory for caching filesystem data (buffer cache), which greatly improves read performance for frequently accessed data.

Multi-Channel and NUMA Behavior: Each CPU's memory channels operate in parallel (in interleaved mode by default), meaning if you populate four identical DIMMs on a CPU (one per channel), the memory controller interleaves addresses across them for higher throughput. If channels are unevenly populated (say one channel has a smaller DIMM), the interleaving might not be uniform, but the system will still manage. The NUMA aspect means that the OS will try to allocate memory on the same node as the running thread when possible (NUMA locality). For a storage hub, one might pin disk I/O threads or filesystems to specific CPUs that have direct access to the disk controller (for example, if a RAID card is on a slot attached to CPU2, it's beneficial if the I/O processing threads run on CPU2 and use CPU2's memory). Linux's NUMA policy can be configured or left to its automatic NUMA balancing.

ECC and Reliability: With ECC memory, the system is well-protected against memory bit flips. The combination of ECC RAM and the chipset's memory scrubbing and monitoring (via ECC sensors) provides enterprise-grade reliability for long-term data integrity. Servers like this often run memory tests and scrubbing in the background. If a DIMM starts to show multi-bit errors (which ECC can't correct), the system can alert the admin via IPMI (the BMC can report ECC errors). For a storage server tasked with safeguarding data, ECC memory is a must-have; the X9QRI-F+ fully supports this, as indicated by both Intel and Supermicro specs ³⁴ ³⁷.

In summary, the memory subsystem of the X9QRi-F+ is *very robust and scalable*. Our configuration of 82 GB, though oddly sized, will still deliver high memory bandwidth (likely running at 1333 or 1600 MT/s due to multi-DIMM channels) and the benefits of ECC. The multi-channel interleaving per CPU and the NUMA distribution across CPUs mean the system can cache and buffer a large amount of disk data – beneficial for a storage hub role. Administrators should be mindful of memory placement: balancing DIMMs and enabling NUMA-aware settings can yield the best performance. But even out of the box, the platform’s memory bandwidth and capacity are well-suited for an enterprise storage server, providing a large page cache for the OS and minimizing disk reads for frequently used data.

4. Storage Architecture: Onboard SATA (No Native SAS) and RAID Considerations

The X9QRi-F+’s onboard storage interfaces are provided by the Intel C602 chipset. The board has **6 SATA ports** in total: **2× SATA 3.0 (6 Gbps)** ports and **4× SATA 2.0 (3 Gbps)** ports ³⁸. These correspond to the C602’s integrated AHCI controller capabilities: typically, C602 (equivalent to Intel X79/SNB-E PCH) offers two 6 Gb/s SATA channels and four 3 Gb/s channels natively ³⁸. **No SAS controller is integrated on the X9QRi-F+**, meaning it does not have native support for SAS drives or hardware SAS RAID out-of-the-box ³⁹. (By contrast, a sibling model X9QR7-TF+ included an LSI SAS controller on board, but the X9QRi-F+ omits this, focusing on SATA only.) Therefore, the storage design for this system must either rely on these 6 SATA ports or utilize add-in cards for more advanced storage needs.

SATA 3.0 vs SATA 2.0: The two SATA 3.0 (6 Gb/s) ports allow full-speed use of modern SSDs and high-performance HDDs. Each SATA 3 port has a theoretical max bandwidth of ~600 MB/s, which is ideal for SSDs that often reach 500+ MB/s sequential throughput. In contrast, the four SATA 2 (3 Gb/s) ports are limited to ~300 MB/s each, which can bottleneck SATA SSDs (which would negotiate to 3 Gb/s and be capped around ~270 MB/s throughput). For spinning hard drives (HDDs), SATA 2 is usually sufficient since most 7.2K RPM disks deliver 100–200 MB/s, but for SSDs or 15K RPM enterprise drives, the SATA 2 ports would constrain peak performance. In a mix of SSDs and HDDs, one would typically put the fastest drives (or any SSD scratch disks, etc.) on the two 6G ports, and use the 3G ports for slower disks or optical drives. It’s noteworthy that having only 2 SATA 6G ports is a limitation for an intended “storage hub,” since one might easily have more than 2 SSDs or high-speed drives.

Aggregate Bandwidth and Chipset Limits: All six SATA ports connect through the Intel PCH, which in turn connects to the CPUs via a DMI link (approximately PCIe 2.0 x4 bandwidth, ~2 GB/s). This bus could become a bottleneck if multiple SATA ports are active at once. For example, if all 6 ports were simultaneously busy (say during a RAID rebuild or heavy I/O), the total throughput could, in theory, approach $2 \times 600 + 4 \times 300 = 2,400$ MB/s (if two SSDs and four HDDs were all maxed). That is roughly equal to ~19.2 Gb/s, nearing the DMI’s ~20 Gb/s limit. In practice, you rarely saturate all drives at once with sequential load, but heavy parallel I/O (like a parity-scan or scrubbing on a multi-disk array) might push the limits. So, the chipset’s own link to the CPU is something to keep in mind – for extreme storage performance, offloading to PCIe-based controllers (which can attach directly to CPU PCIe lanes) is often preferable. Additionally, because only one of the four CPUs is directly connected to the C602 PCH (likely CPU1), any other CPU accessing the SATA drives must communicate via QPI to CPU1, adding overhead. This again suggests that a storage-heavy workload should be managed in a NUMA-aware way (e.g., run SATA-based storage services on the CPU attached to the PCH).

RAID Options (Chipset RAID vs Software vs Hardware): The C602 chipset does offer Intel Rapid Storage “Enterprise” RAID (RSTe) for the SATA ports, supporting RAID levels 0, 1, 5, 10 in firmware ⁴⁰ ³. In the specification, this is noted as “RAID 0,1,5,10” for both SATA2 and SATA3 ports ³. However, this is essentially a firmware-assisted software RAID (also known as “fake RAID”) – it requires specific Intel drivers and is typically used on Windows. For a Linux-based storage hub, one would rarely use the motherboard’s firmware RAID due to limited support; instead, Linux’s own software RAID (MDADM) or a software-defined storage solution (like ZFS) would be used. **Linux MD RAID** can easily handle RAID 0/1/5/10 across these SATA ports, leveraging the CPU for parity calculations in RAID5/6. With four powerful Xeon CPUs, the overhead of software RAID is not an issue – the system has plenty of processing power for computing parity or rebuilds. In fact, software RAID may outperform older hardware RAID cards in some cases, because it isn’t constrained by a fixed XOR engine and can use all available CPU cores for parallel computation (especially relevant for RAID6 or ZFS’s RAID-Z which involve heavier math like Reed-Solomon or checksums).

That said, using only the 6 onboard SATA ports limits the number of drives. The user scenario envisions a “mix of SSDs and HDDs in RAID configurations.” With only 6 ports, one could at most have, say, a RAID-10 of 6 drives or a RAID-5 of 6 drives. 40 TB raw (as mentioned in an example on forums) would be difficult with 6 drives unless using very large disks (e.g., 6×8 TB in RAID-5 gives ~40 TB usable, but that’s pushing it and rebuild times become worrying with large SATA drives and no hot spares). Indeed, community feedback suggests “Only 6×SATA is not enough... The OP wants to start with a 40 TB volume in RAID5. That won’t work out too well with only 6 SATA ports” ⁴¹. The limitation is both capacity (number of spindles) and performance (few spindles mean less aggregate IOPS and throughput). Therefore, a serious storage build would look to **expand storage connectivity via add-on controllers**.

Recommended RAID/HBA Add-on Cards: The X9QRI-F+ provides four PCIe slots (see Section 5), which can be used for storage controllers. For a storage-centric server, a common approach is to add an **HBA (Host Bus Adapter)** or a **hardware RAID card** to gain more ports (often SAS/SATA ports) and possibly better performance. Given modern trends, many prefer an HBA with lots of ports (and then do software RAID or ZFS) rather than hardware RAID cards. A popular choice is an **LSI SAS HBA**, such as the LSI 9207-8i (which provides 8 internal SAS/SATA 6G ports on a PCIe 3.0 x8 card). The LSI 9207-8i uses an SAS2308 controller and can handle 8 drives at full SATA 6G speed without bottlenecking a PCIe 3.0 x8 slot ⁴². In fact, it’s noted that “An LSI 9207-8i card is the way to go, it’s PCIe 3.0 x8 and will support 8 drives with ease”, whereas older cards on PCIe 2.0 x4 could become a bottleneck ⁴³. The 9207-8i or its equivalents (SAS 2308-based HBAs) are widely recommended as solid choices for building storage arrays ⁴⁴ ⁴⁵. They support both SAS and SATA drives, giving flexibility to use enterprise SAS disks or expand beyond the 6 onboard SATA limit. Another similar card is the LSI 9211-8i (older SAS2008-based, PCIe 2.0 x8), which also provides 8 ports but on PCIe 2.0 (with slightly lower aggregate bandwidth). If even more drives are needed, one can use multiple HBAs or a higher-port count HBA (like an LSI 9300-8i for SAS3 12 Gbps, or 16-port variants). According to a ServeTheHome guide, the SAS2308 HBAs (e.g., LSI 9207) are still considered “good” options for <8-drive setups, though newer SAS3 (3008/3108) HBAs offer higher performance for SSD-heavy arrays ⁴⁶. For mostly HDD usage, the 9207-8i is plenty: it can connect 8 drives and deliver full 6 Gbps to each. With two such cards, one could have 16 direct-attached drives, which is often more than enough for a storage server enclosure (and this board has enough PCIe slots to accommodate that).

If one prefers a **hardware RAID controller** (with on-board cache and XOR engine, etc.), options include LSI/Avago MegaRAID cards (e.g., LSI 9271-8i for 6G SAS or 9361-8i for 12G SAS). These provide battery-backed cache and can offload parity calculations. However, using hardware RAID in Linux has pros and cons. Modern practice, especially with Linux and ZFS, often steers away from hardware RAID in favor of either

pure HBA (IT mode) or software solutions. In fact, ServeTheHome explicitly suggests *“Skip them. No really, skip [hardware RAID] cards”* for ZFS/NAS builds, citing that they add complexity and their benefits are unnecessary when the CPU is strong ⁴⁷. If the use-case is a traditional RAID5/6 with a need for write-back cache, a hardware RAID card with BBU might still be attractive – but one must ensure compatibility and driver support in Linux (MegaRAID cards are supported via `megaraid_sas` driver, for example). For a pure Linux storage hub, a safer route is typically: HBA + software RAID, or HBA + ZFS. This leverages the robust error-handling and flexibility of software solutions, and with four Xeon CPUs, the computational overhead is trivial (the system has vector instructions and multiple threads to handle RAID parity very quickly).

Storage Bandwidth Considerations: With a suitable HBA, the system can overcome the DMI bottleneck of the PCH. For instance, an LSI 9207-8i (PCIe 3.0 ×8) has up to ~8 GB/s bandwidth to the CPU ⁴⁸, which far exceeds the ~2 GB/s from the onboard SATA. To give a concrete performance example: in one test, **eight SATA SSDs** (Crucial M4 drives) connected to an LSI 9207-8i achieved about **4.1 GB/s** aggregate throughput in sequential reads ⁴⁸ ⁴⁹. This clearly broke the ~3 GB/s limit that older PCIe 2.0 controllers had, demonstrating the advantage of a PCIe 3.0 HBA for high-bandwidth storage arrays ⁵⁰ ⁴⁸. In our context, 4.1 GB/s is **far above** what dual Gigabit LAN can handle – thus if the storage server is to actually serve that data out, network would be a bottleneck (we’ll address networking in Section 6). But it shows that internally, the system can move data at multi-GB/s rates if the storage subsystem is designed properly (e.g., using HBAs and possibly RAID-0 or multiple RAID groups). For HDD arrays, the throughput will be lower (e.g., a RAID-0 of 8 HDDs might get ~1 GB/s sequential), but still the principle holds: the PCIe-attached controllers remove chokepoints.

Another aspect is **SSD vs HDD mix**: If using a mix, one might dedicate SSDs as a cache tier (L2ARC or flash cache) and HDDs as capacity tier. The absence of NVMe on this older platform means SATA/SAS SSDs are the go-to for flash storage. One could also use SAS expanders or JBOD enclosures via an HBA external port (if needed to attach many drives externally). The board, not having onboard SAS, really relies on those expansions for scaling beyond 6 drives.

Summary of Recommendations: Use the two onboard SATA 6G ports for two of your fastest SSDs (perhaps for boot drives or a small RAID-1 for the OS, or for caching). Use the four 3G ports for slower disks or not at all if not needed. For the main storage arrays, **add a RAID controller or HBA**. A good approach is an HBA like the LSI 9207-8i (8-port, SAS2 6G) or, if planning lots of SSDs or just to be future-proof, a LSI 9300-8i (8-port, SAS3 12G) – though on this DDR3 platform, SAS3’s 12 Gbps per port is only fully useful if you have very fast SSDs or NVMe (which you can’t directly use here). Many homelab and enterprise users have gravitated to these LSI HBAs; they are known to handle many drives well and have open-source drivers. As one user succinctly put it: *“9211 and 9207 are two popular ones if you are only running mechanical storage”* ⁵¹ – meaning they suffice for HDDs. And another said the 9207-8i **“will support 8 drives with ease”**, whereas a PCIe 2.0 card might “barely support 8 drives” in terms of bandwidth ⁴².

For **RAID levels**: if using Linux MD or ZFS, the CPU can handle RAID5/6 parity calculation across many drives – the multiple cores can work in parallel on stripes. The E5-4600 v2 also have AVX and XOR optimizations which Linux RAID can leverage (there are optimized RAID5 XOR implementations that use SSE/AVX). So even with, say, 8 drives in a RAID6, this system could likely calculate parity at hundreds of MB/s without breaking a sweat on CPU usage. A hardware RAID card would offload that, but again at the cost of another layer and potential single point of failure (if the RAID card fails). ZFS would use a different approach (copy-on-write, checksums for all data). If ZFS is used, one would definitely use an HBA (IT mode) so the drives are passed through; ZFS would then use the ECC RAM (82 GB is great for ZFS ARC) to cache and ensure data integrity.

Disk Connectivity Summary: The onboard SATA is suitable for a small number of drives and can be used for boot or a simple array, but for a “bare-metal Linux storage hub” with a mix of SSDs/HDDs and serious capacity, an expansion HBA/RAID card is strongly recommended. It overcomes the port count limitation and performance limits of the chipset. Popular, well-supported choices include the LSI SAS 2308 series HBAs (like 9207-8i), which provide ample bandwidth and are known to work flawlessly under Linux with modules like `mpt3sas`. Moreover, because the board has multiple PCIe 3.0 x16/x8 slots, adding such controllers (even multiple) is feasible without saturating the CPU’s PCIe capabilities (each CPU has 40 PCIe 3.0 lanes, see Section 5, and we can distribute controllers across different CPUs’ lanes to balance load).

In conclusion, **the absence of onboard SAS** means the X9QRi-F+ is somewhat barebones for storage out-of-the-box, but its expandability via PCIe more than compensates. With the right controller cards, the system can accommodate high-throughput RAID arrays. Administrators should plan the storage layout to utilize the two 6 Gbps ports for critical high-speed drives, and leverage PCIe cards for bulk storage. With Linux, software RAID or ZFS is a robust choice, taking advantage of the system’s CPU and memory strengths.

5. Expansion Capabilities: PCIe Slots for RAID Controllers, NICs, etc.

Despite the large footprint of four CPU sockets and 32 DIMMs, the X9QRi-F+ provides a reasonable set of expansion slots. It has a total of **4 PCI Express 3.0 slots**, arranged as:

- **2 × PCIe 3.0 x16** slots (full-length, 16-lane slots)
- **2 × PCIe 3.0 x8** slots (physical x16 connectors wired with 8 lanes) ⁵² ⁵³ .

This layout gives flexibility to install high-bandwidth devices. Typically, in a quad-socket board, the slots are electrically connected to different CPUs to distribute I/O load. The X9QRi-F+ likely ties each slot’s lanes to a specific CPU’s PCIe controller. (The exact mapping isn’t in the short spec, but often, CPU1 might feed Slot1 x16 and Slot2 x8, CPU2 feeds Slot3 x16, CPU3 feeds Slot4 x8, etc., or some similar scheme. We do know from a forum that “socket #4 controls the onboard LAN ports” ⁴ , which implies the NIC is on CPU4’s PCIe lanes; by analogy, some slots could also be attached to particular CPUs.) What this means is that to use all slots, **all four CPUs must be installed**. If one or more CPU sockets were empty, some PCIe slots would not function. In our case, with four CPUs in place, all slots are active and each can operate at PCIe Gen3 speeds (8 GT/s per lane).

Bandwidth and Use Cases: The two x16 slots each provide up to ~16 GB/s of bandwidth (in each direction) at PCIe 3.0. The x8 slots provide ~8 GB/s each. These figures are ideal for high-speed peripherals: - A RAID or HBA card can be placed in an x8 slot (most such controllers are 8-lane cards) and get full bandwidth. For instance, an LSI 9207-8i HBA (PCIe 3.0 x8) will saturate an x8 slot and can push those aforementioned 4 GB/s of data ⁴⁸ . - If one uses a high-end RAID card that’s x8 or x16, there’s room. Some RAID cards with more ports (e.g., 16-port HBA) might use x8 and still be fine. - A 10 Gigabit Ethernet (10GbE) NIC typically uses PCIe 3.0 x8 (or x4 for single-port). Dual-port 10GbE adapters often are x8. These could be installed to greatly expand network throughput (since the board only has 1GbE onboard). With two x16 and two x8 slots, one could conceivably add, for example: one 10GbE (or even 40GbE) NIC, one or two storage controllers, and still have a slot left for another accessory (like a GPU for compute or an NVMe adapter, etc.).

Electrical and Physical Considerations: The slots are full-length (x16 physical), which means you can plug larger cards even into the x8-wired slots. The board being proprietary EATX size means a lot of space; likely, these slots are arranged spaced out to fit possibly double-width cards (if one wanted to add a GPU or an

FPGA card, for instance). However, in a storage server scenario, one usually uses low-profile RAID/NIC cards (especially if it's mounted in a 2U chassis). The power supply to these slots would come from the 12V plane, and the board can supply the standard 75W per slot; if more power is needed (like a GPU requiring 300W), that would involve auxiliary PCIe power connectors from the PSU (GPUs in a server board scenario are uncommon but not impossible – some might use this platform for HPC as well).

One should note that while many lanes are available, if a single CPU ends up hosting multiple high-throughput cards, it could overload that CPU's I/O or QPI link to others. It can be wise to distribute add-in cards between CPUs. For example, if CPU2 and CPU3 each have a big device, then any cross-traffic might use QPI between them rather than all funneling through one CPU. The good thing is, with four CPUs, the total available PCIe 3.0 lanes is $4 \times 40 = 160$ lanes (theoretically). The board doesn't expose all of those (only 48 lanes via slots, plus some used for onboard devices like NIC, IPMI, etc.), but it means none of the slot configurations are constrained by lack of lanes. Each x16 likely is using a full x16 from one CPU, each x8 using 8 lanes from another CPU, and still many lanes per CPU are unused or used for onboard I/O (for example, video BMC, NICs, etc. might consume a few lanes from one CPU or PCH).

Expansion for Storage: As discussed in Section 4, one likely use of these slots is adding storage controllers (SAS HBAs, RAID cards, or even NVMe adaptors). The board does not have M.2 or U.2 connectors (those became common in later-gen boards), so if one wanted to use NVMe SSDs, they could via a PCIe adapter card (e.g., a PCIe NVMe drive or a bifurcation card like the ASUS Hyper M.2 if supported). However, it's unclear if the X9 generation BIOS supports NVMe boot; NVMe came after this era, so boot support might be limited, but as a data drive it could work if BIOS passes it (perhaps via UEFI driver). That said, focusing on SATA/SAS, the PCIe slots definitely allow adding e.g. a **16-port HBA** (which often is basically a x8 card with an expander on it or dual controllers). If someone needed huge drive counts, one could also use those slots for **external SAS HBAs** (with external SFF-8088 ports) to connect to a JBOD chassis full of drives.

Expansion for Networking: The onboard networking is dual GbE (which is modest). If the storage hub needs to serve lots of data, adding a faster NIC is advisable. The PCIe slots can easily host: - A **10G Ethernet** card (e.g., Intel X520 or X540 dual-port 10GBase-T). Those are typically PCIe 2.0 or 3.0 x8. Installing one in any of the x8 or x16 slots would give an order of magnitude more bandwidth than the built-in 1G ports (10 Gb/s vs 1 Gb/s). Dual 10G NICs can even approach 20 Gb/s aggregate (if using both ports). - If extremely high network throughput is needed, **40 GbE or 56 Gb Infiniband** cards could be used. A single 40GbE QSFP+ NIC often uses PCIe 3.0 x8. This platform could support that. Four CPUs can definitely drive multiple 10G links without CPU saturation, especially since networking tasks can be spread across cores (and the Intel NICs or high-end NICs support multi-queue and offloads). - Additional Gigabit ports (like a quad-port GbE card) if needed for redundancy or connectivity to multiple networks can also be added. With two x16 and two x8, one could conceivably add, for instance, *both* a 10GbE card and a second RAID card, etc.

Other Add-ins: Other expansions might include things like a SAS expander card (if not using an HBA, one could use an expander backplane in a chassis – some Supermicro chassis listed in Section 7 have expander backplanes, which would attach to an HBA). Also, one could add **NVMe storage** via a PCIe SSD or adapter – e.g., an Intel DC P3700 PCIe SSD (which is a full add-in card). The x16 slot would accommodate such an NVMe drive giving direct high-speed flash storage (though integration into a software RAID might require using it separately, since NVMe wouldn't mix with SATA in the same mdadm RAID easily).

Compatibility and Throughput: The key advantage here is that **PCIe 3.0** is supported by these Ivy Bridge CPUs ⁵⁴. This platform was among the first to bring PCIe Gen3 to multi-socket systems, so it's not lacking

in I/O speed (unlike older Nehalem/Westmere systems that had Gen2). As long as devices are PCIe 3.0 capable, they'll run at full speed. If you install an older Gen2 card, it will run at Gen2 speeds (which is fine in some cases, e.g., a PCIe 2.0 x8 card still gives ~4 GB/s, which is enough for many uses). Given that each CPU's 40 lanes might not all be used by the slots, sometimes boards like this also have additional onboard devices – in this case, onboard devices are just dual NIC (which could consume e.g. a single x4 link from one CPU or via PCH) and maybe a dedicated IPMI interface via the BMC (which usually connects via a low-bandwidth interface, not PCIe). So the majority of PCIe lanes are allocated to the slots, meaning minimal contention.

One caveat: If heavy I/O is concentrated on one CPU's lanes, it could saturate that CPU's link to the others if cross-node memory or CPU involvement is needed. For instance, if a RAID card on CPU2's lanes is being accessed by a thread on CPU1, data goes across QPI. The QPI link (8.0 GT/s) has finite bandwidth (~32 GB/s raw, maybe ~12.8 GB/s effective each direction for data after overhead). Usually that's plenty even for a 4 GB/s stream, but it's something to consider (i.e., for best performance, use local CPU affinity for I/O tasks, as mentioned before).

Electrical/Mechanical Fit: The board, being large, likely requires a 4U or special 1U/2U chassis (Supermicro's doc lists optimized chassis including a 4U with many drive bays ⁵⁵). In a 4U chassis, one can use full-height, full-length cards. If this is in a 2U, often risers or right-angle connectors are used. The board spec doesn't explicitly mention OCP slots or anything beyond those 4, so all expansions must happen through those standard PCIe slots.

In conclusion, the X9QRi-F+ offers **ample expansion for a storage server**: you can populate it with high-speed RAID/HBA controllers, upgraded NICs, or other accelerators. A balanced approach might be: - Use one x16 slot for a dual-port 10 GbE NIC (ensuring the storage server isn't bottlenecked by networking). - Use one x8 slot for an 8-port SAS HBA to connect a large disk array. - Use another slot for a second HBA or a different function (maybe a backup 10GbE or a Fibre Channel card if connecting to SAN, etc.). - Keep one slot spare (or use for an NVMe SSD card for fast local scratch storage or metadata caching).

This flexibility is particularly useful since the board itself lacked advanced storage controllers – it expects the user to tailor it via expansions. It is also forward-looking enough that even today one could install relatively modern expansion cards (the PCIe standard is still current). So the **suitability for adding RAID controllers, NICs, or HBAs is excellent**. There are enough slots to integrate all the components needed to turn this into a feature-rich storage appliance (RAID arrays, high-speed network interfaces, etc.). The key is to plan out which slot for which purpose, possibly aligning them with CPU NUMA nodes for efficiency (for example, assign the storage HBA and the processing threads for it to the same CPU).

6. Networking and Management: Dual Intel i350 Gigabit & IPMI 2.0

Onboard Networking: The X9QRi-F+ includes an onboard **Intel i350 dual-port Gigabit Ethernet** controller ⁵⁶ ⁵⁷. The Intel i350 is a server-grade NIC, providing two 1 GbE RJ45 ports on the rear I/O. The i350 supports features like VLAN tagging, NIC teaming, checksum offloads, and virtualization enhancements (e.g., VMDq for VM traffic separation) ⁵⁸ ⁵⁹. Each port can run at 10/100/1000 Mbps. This dual-port NIC is often used for redundancy or bonding – for instance, one could aggregate them (802.3ad link aggregation) to get up to 2 Gb/s throughput (though a single TCP flow is still 1 Gb/s unless using multiple streams). In an enterprise environment, these might be configured in active-backup mode for high availability or connected to separate switches. The Intel i350 is known for its reliability and performance at the 1 GbE level, and it has

well-supported drivers (the Linux **igb** driver covers the i350 family ⁶⁰). Offloading features and interrupt moderation in the i350 help it handle high packet rates with low CPU usage, which is useful if the storage server is serving many small files or network IOPS.

That said, **Gigabit Ethernet will likely be the primary bottleneck** in a modern storage server scenario. Even with both ports aggregated, ~2 Gbps (250 MB/s) is far below what the disk subsystem can do (as we saw, a single SSD can reach 500 MB/s, and our potential RAID setups can be in the gigabytes/sec). Thus, if this storage hub is intended to push high throughput (e.g., for backups, virtualization storage, or large file transfers), one would strongly consider adding faster network interfaces via PCIe (discussed in Section 5). The presence of only 1GbE onboard is reflective of the era (2012–2013) when 10GbE was still optional or on higher-end “-TF” models (indeed, Supermicro’s X9QR7-TF+ variant had 10GBase-T ports onboard). For our X9QRi-F+, upgrading networking is a recommended step. A dual-port 10G card could offer ~20 Gbps, and even a single-port 10G (which can do ~1 GB/s) would unleash a lot more of the storage system’s performance to clients.

Remote Management (IPMI): A standout feature for enterprise boards is the built-in **IPMI 2.0 BMC** (Baseboard Management Controller). The X9QRi-F+ integrates a **Nuvoton WPCM450 BMC** which provides IPMI functionality with **KVM-over-LAN** and virtual media support ¹¹. This means the server can be managed remotely regardless of its power state or OS condition. There is a dedicated RJ45 port for management – the spec lists “1× Realtek RTL8201F PHY (dedicated IPMI LAN)” ⁶¹ ⁶². The Realtek RTL8201 is a 10/100 Mbps PHY that connects to the BMC, indicating the IPMI port is Fast Ethernet (common for IPMI, as high bandwidth isn’t needed for console and management). This port allows one to plug the server into a management network and perform actions like power on/off, BIOS access via remote KVM console, and mounting of virtual ISO images for OS installation. It’s essentially like having a remote console and remote DVD drive via the network – extremely useful for headless operation in a data center or homelab rack. The BMC also monitors sensors (temperatures, fan speeds, voltages) and can send alerts (via SNMP or email, etc., depending on configuration).

The **IPMI and BMC** also handle the onboard video: the WPCM450 has an integrated Matrox G200e graphics core (with 16 MB VRAM typically) ¹⁰, which outputs to the rear VGA port and to the KVM remote console. This is not for high performance video, but purely for server maintenance (displaying BIOS, text console, and maybe basic GUI in OS if needed). It frees the need to have a GPU for basic output. The BMC and IPMI can be accessed using utilities like IPMITool or through a web interface (Supermicro’s IPMI web GUI) – allowing remote monitoring of system health and even launching a Java or HTML5 KVM session to view the screen.

Networking Features: The Intel i350 NICs support advanced features like jumbo frames (up to 9KB MTU), multiple transmit/receive queues, and on some models, SR-IOV (single-root I/O virtualization) which allows VMs to directly access portions of the NIC. The i350 (being dual-port) also supports features useful for iSCSI or NFS storage serving, such as TCP segmentation offload and IEEE 1588 time stamping. If this storage hub serves as e.g. an iSCSI target, the i350 will handle it well at Gigabit line rates. But again, for higher performance (like serving many VMs over NFS or iSCSI), 10GbE is recommended. The i350 is recognized by Linux out-of-the-box; drivers for it have been in the kernel for ages (and Intel also provides updated driver packs if needed).

Remote Management Capability: With IPMI, one can do things like monitor the system’s temperature sensors, check PSU status, and even access the BIOS remotely. This is crucial for an enterprise-style storage

server which might be located in a server room or colocation. It means issues can be diagnosed and fixed (or firmware updated) without physical presence. IPMI also often exposes a watchdog timer and can automatically reset the system if it hangs.

It's worth noting that the IPMI uses its own LAN interface (the dedicated port) and also might have a "shared" mode where it can piggyback on one of the Intel NIC ports if configured that way (some Supermicro boards allow the IPMI to share NIC1 when no dedicated cable is plugged, but since this board has a dedicated Realtek PHY, it likely uses that exclusively by default). The BMC itself runs on an ARM or 8051-based microcontroller within the WPCM450; it has its own firmware which can be updated. In terms of security, IPMI should be on a trusted network (and have strong passwords) because it has deep control over the system.

Summary: The X9QRi-F+ provides basic **dual Gigabit networking** suitable for management, low-volume file serving, or redundancy, but to fully exploit the storage potential, faster networking should be leveraged via expansion cards. On the management side, the **IPMI 2.0** capability is a big plus for a "bare-metal Linux storage hub" – it means even if the OS is down, you can remotely reboot or troubleshoot. Many enterprise users would consider IPMI essential for a headless storage server in production. It effectively allows the server to be treated like a managed appliance.

In practice, one might use the onboard dual GbE for secondary purposes even after adding a 10GbE card – for example, one port could be reserved for management or monitoring traffic, or for a separate subnet for replication traffic, etc., while the add-in NIC handles primary client data. The possibilities for network configuration are broad. But the key point: **onboard = 2×1GbE (Intel i350)** ⁵⁷, **management = 1× IPMI LAN (Realtek 100Mb)** ⁶². For anything above a couple hundred MB/s of throughput, plan to use the PCIe slots for network upgrades (which the system has room for).

7. Thermal and Power Considerations: Cooling Four CPUs and Powering the System

Running four high-end Xeon CPUs and a fleet of drives demands careful attention to power and cooling in the system design. The Supermicro X9QRi-F+ is often deployed in server chassis specifically designed for quad-socket boards, which include robust cooling and large power supplies. Let's break down the considerations:

Power Supply Requirements: With four CPUs at up to ~115 W TDP each, that's ~460 W for the CPUs alone under full load. Add to this the memory power: DDR3 RDIMMs can draw around 2–5 W each (depending on capacity and activity). With 32 DIMM slots populated, worst-case could be on the order of ~100 W just for memory (less in our 82 GB config since not all slots are full). Each spinning hard drive might consume ~5–10 W (particularly during spin-up), and each SSD maybe 2–5 W. A RAID or HBA card draws ~10–15 W typically, and a 10GbE NIC around 5–15 W depending on model. Fans and the motherboard chipset/BMC also consume some power. Summing these, it's easy to reach **700–800 W under load** in a fully populated system (especially if many drives are attached). Consequently, an appropriate PSU is needed. Supermicro's recommended chassis for this board come with **high-wattage, server-grade PSUs**: for example, a 4U tower chassis (SC748 or SC848 series) with a **1400 W or 1600 W** redundant power supply ⁵⁵. Indeed, the spec sheet lists compatible Supermicro chassis like the SC418, SC748, SC848 with "R1K62" or "R1K43" PSUs

(meaning 1620 W, 1400 W units) ⁵⁵. These PSUs are typically 80 Plus Platinum or Gold efficiency, delivering strong 12V rails to feed the board and peripherals.

For a stable storage server, it's common to use **redundant PSUs** (two hot-swap PSUs in an ORing configuration), so that if one fails, the other takes over. The models mentioned (e.g., 8047R-TRF+ SuperServer) likely had dual 1+ kW PSUs ⁶³. While not strictly necessary for function, redundancy is key for availability in enterprise settings.

Thermal Profile: Four CPUs generate a lot of heat in a confined space. The layout on the board likely has two pairs of sockets. In many quad-socket boards, the CPUs are arranged in a square or rectangle, which can create challenges for airflow – the middle ones might run hotter if airflow is not direct. Supermicro's guidance often includes using *air shrouds* to channel airflow from fans directly over the CPU heatsinks ⁶⁴ ⁶⁵. In the spec snippet, they mention specific air shroud part numbers for 1U and 4U chassis (e.g., MCP-310-74805-0B for SC748) ⁶⁶, underscoring that directed airflow is needed.

Cooling Solution: The CPUs themselves would use either passive heatsinks (if in a chassis with high-speed fans forcing air through) or active heatsinks (with dedicated fans on each) depending on the chassis. In typical rackmount deployments, especially 1U/2U, they use passive blocks with strong chassis fans (often high RPM screamers) to move air front-to-back. The thermal design of the chassis ensures each CPU and DIMM bank gets adequate cooling. Given T_{case} ~75°C for these CPUs, one wants to keep inlet temps low and have a good exhaust of hot air. **Fan redundancy** is also often present (multiple fans in case one fails).

In a tower or open-air environment, one could use active heatsinks (each with a fan) as long as there's enough case airflow to evacuate heat. However, four CPUs with fans can be loud and also need clear airflow paths.

Additionally, the board's VRMs (voltage regulator modules) for the CPUs and memory are under heavy load delivering potentially >100 A current to each CPU at sub-2V. Those VRMs (typically near each socket) also need cooling – which they get from the same airflow stream or dedicated heatsinks. The chipset (C602 PCH) has its own thermal needs, though C602 is not extremely high TDP (it might be around 7–10 W, often with just a small heatsink).

Quad-CPU Thermal Concerns: One consideration is that in a cramped chassis, the combined heat could lead to thermal throttling if cooling is insufficient. Each E5-4600 v2 has built-in thermal management (if it hits thermal limits it will throttle frequency or even trigger protective shutdown). Ensuring that doesn't happen under sustained load is a matter of using the right cooling solution. That's why specific chassis are recommended by Supermicro. For example, they list a 1U chassis (818A) but note "rev. M required" which likely means a certain improved revision for those high TDP chips ⁶⁷. Usually, 4 CPUs in 1U is pushing it – they likely would use lower-TDP CPUs (like E5-4657L v2 at 115W might even be too high for 1U without noise). A 2U or 4U chassis is more forgiving. The SC848 mentioned is a 4U 48-bay storage chassis, which definitely has big fans and can handle heat but it's huge.

Airflow Management: The concept of the *air shroud* is important. It's a plastic guide that sits over the board directing front fan air through the CPU heatsinks and DIMMs, rather than letting it spill around. In multi-socket systems, these dramatically improve cooling efficiency. If the user is setting this up in, say, a rackmount case, they should ensure to use the proper shroud (or in a custom setup, emulate that directed flow). In a standard tower case (if one attempted that), cooling 4 CPUs could be challenging; typically an off-

the-shelf tower might not have mounting for such a large board or the airflow aligned for four separate hot spots.

Noise and Environment: A fully cooled system with 4 CPUs and many drives will be *loud*. High CFM fans (sometimes 8k+ RPM) are often needed. This is fine in a server room, but a consideration if this is repurposed in a non-datacenter environment.

Heat from Other Components: We must also consider that each add-in card (HBA, NIC) dissipates heat, and drives, especially 15k SAS drives, can run hot. Thus, the chassis needs good front-to-back airflow across all drive bays. Typically, storage chassis have fan walls right behind the drive backplane to suck air through the drives and blow it over the motherboard.

Power Draw in Use: While the TDPs are high, actual power usage will depend on workload. A storage server often has bursts of activity but may be relatively idle at times (drives spun down or waiting). Idle power of this system could still be a few hundred watts (since each CPU at idle might drop to, say, 20W, memory maybe 20W, etc., plus base overhead). Under heavy load (like stress-testing CPU + all drives busy), the PSU capacity ensures stable delivery. A too-small PSU would risk overloading on spikes (especially spin-up, where all drives simultaneously draw extra current).

Power Delivery on Board: The board likely has 2 x 8-pin EPS connectors (or even 3) to feed CPU power, plus the 24-pin ATX. All must be connected to supply enough current. PSUs for such boards usually have multiple EPS12V plugs.

Thermal Monitoring and Management: The BMC monitors temperatures of CPU, system, etc. If a fan fails or a temp gets high, it can ramp up remaining fans (Supermicro IPMI has automatic fan control modes). Administrators can configure thresholds to get alerts if, say, a CPU goes above 85°C or a drive temperature is too high. This helps proactively manage cooling (for instance, increasing data center AC if needed).

In summary, a quad-CPU storage server requires **significant cooling** and a **robust PSU**: - **PSU:** Use ~1000 W or higher, preferably redundant. In an example configuration, a Supermicro 4U chassis with dual 1620 W PSUs was suggested ⁵⁵, which gives ample headroom even if all components draw max power (and redundancy means each PSU can carry the load if the other fails). - **Cooling:** Ensure strong front-to-back airflow. If using Supermicro's chassis, follow their cooling recommendations (correct fan models, install the air shroud). If a custom chassis, populate it with high-speed fans and consider the use of ducting to channel air over the CPUs and RAM. - **Thermal compound and heatsinks:** When installing CPUs, proper thermal paste and fully tightening the heatsinks is critical. Because one runaway CPU can overheat if not mounted right, causing throttling or shutdown. - **Environmental conditions:** Keep the server in a cool ambient environment (datacenter ~20-25°C). The specs likely allow operation up to ~35°C ambient, but life is better for electronics at cooler temps. Also factor in that 4 CPUs and many drives will dump a lot of heat into the room (maybe 2000–2500 BTU/hr or more), so adequate AC is needed.

The operational outcome should be that even under heavy load (all drives busy, CPU calculating parity or serving data), the CPU temperatures stay below throttling thresholds (ideally < 70°C), and drive temperatures stay in safe ranges (< 50°C for HDD, < 70°C for SSD). The PSU should not be near its limit to avoid stress – aiming for ~50-60% load at typical operation is good for efficiency and longevity (PSUs are most efficient around that range).

In practice, Supermicro's system integration (like the **SYS-8047R-TRF+** which corresponds to this board ⁶⁸) would have validated that with the recommended chassis and cooling, the system runs reliably. Therefore, using those guidelines (or similar spec components) will ensure thermal stability. Failing to cool properly could result in throttled performance or, worst-case, hardware failures (CPUs will shut off if overheated severely).

8. Linux Compatibility and Management: Chipset, Drivers, and Configuration

One of the advantages of using slightly older, well-established hardware like the X9QRi-F+ platform is that **Linux support is very mature**. Virtually all components on this board are supported by mainstream Linux kernels without requiring special drivers:

- **Chipset (Intel C602):** This chipset uses the standard Intel PCH architecture similar to X79/C600 series. The SATA controller is an AHCI controller, which the Linux kernel supports out-of-the-box via the `ahci` driver. AHCI mode allows Linux to see all 6 SATA ports and use them normally (in fact, it's recommended to keep the SATA mode in BIOS as AHCI, not RAID, if one plans to use Linux mdraid or ZFS, to avoid any BIOS RAID metadata interference). The USB 2.0 ports (the board has USB 2.0 only, no USB 3.0 on this chipset) are handled by the `ehci_hcd` driver typically. The PCIe root complexes on each CPU and the DMI link are transparent to the OS – devices just show up on the PCI bus.
- **Network (Intel i350):** The i350 NIC is supported by the `igb` driver on Linux ⁶⁰. Kernels in the 3.x, 4.x, 5.x series all include support for i350. For example, Intel's driver release notes show i350 support has been around for a long time. Most enterprise Linux distros (RHEL, Ubuntu LTS, SLES) have the appropriate driver built-in. In Linux, the dual ports would typically show up as `eth0` and `eth1` (or `eno1`, `eno2` under systemd naming) and Just Work™ at gigabit speeds with all offloads available. No manual driver compiling is needed unless one wants the absolute latest Intel out-of-tree driver for some reason. Managing the NIC (like teaming or VLANs) can be done through standard Linux networking tools (bonding driver for teaming, or using NetworkManager/netplan, etc. to configure as needed).
- **IPMI BMC (Nuvoton WPCM450):** The BMC itself operates independently of the OS, but Linux can interact with it via the IPMI interface. Typically, one can load the `ipmi_si` (System Interface) and `ipmi_devintf` kernel modules to communicate with the BMC from the OS (for example, using `ipmitool` to read sensor data or perform chassis commands). This is optional; you can also just access the BMC via the network. The VGA output from the BMC (Matrox G200) can use the simple `ast` driver (if WPCM450 is similar to ASPEED AST2300 family; though WPCM450 is actually an older Winbond BMC with built-in Matrox core – Linux might see it as a generic framebuffer device or a standard VGA). Usually for server, one doesn't rely on the GPU beyond console, so the basic VESA driver suffices for text console.
- **Storage Controllers:** As mentioned, the onboard SATA is AHCI (supported). If a hardware RAID card or HBA is added, Linux has drivers for all popular ones. For instance:
 - LSI SAS HBAs (9211/9207/9300 etc.) use the `mpt2sas` or `mpt3sas` drivers. Those are in-kernel for years.

- LSI MegaRAID controllers use the `megaraid_sas` driver (also well-supported).
- Other possible cards like Adaptec RAID, etc., have their drivers, but LSI is most common and well-supported.
- Intel RSTe (the chipset RAID) can be accessed by Linux via the `isw` (Intel Software RAID) metadata if using `mdadm` (`mdadm` can assemble Intel Matrix RAID), but generally it's not recommended. It's better to use Linux's own RAID from the start.
- **Intel C602 Specifics:** The C602 also includes features like Intel RSTe (we covered), and possibly an SCU (Storage Control Unit, a separate controller in some C60x chipsets) – but I believe C602 specifically just has 6 AHCI ports. No extra SAS unless it was the C606 or C608 variant. Either way, Linux would handle it similarly (e.g., C606 SAS would use an `mptsas` driver if present, but on X9QRi-F+ there's none). The chipset's PCIe lanes (which are PCIe 2.0) might host the Intel i350 NIC (it's possible the NIC is hanging off the PCH's PCIe, because i350 is often a PCIe 2.1 x4 device; or it could be on CPU4's PCIe as per earlier note). Regardless, Linux just sees a PCI device and uses the appropriate driver.
- **System Management under Linux:** With IPMI, one can use `ipmitool` for management tasks. For instance, `ipmitool sensor` will retrieve all sensor readings (temps, voltages, fan speeds) via the BMC, which is useful for monitoring in scripts. There's also `ipmitool lan set` commands to configure the BMC networking from the OS side if needed. The board is also compatible with monitoring software like **Supermicro SuperDoctor** (they have a Linux version) which can show health info, though often it's easier to just use standard tools (`lm_sensors` can pick up some sensor data if configured, or IPMI as mentioned).
- **NUMA and CPU Affinity:** Linux will recognize the 4 NUMA nodes (assuming a modern distro or kernel, NUMA is enabled). Commands like `numactl --hardware` will show 4 nodes, each with certain amount of memory (maybe uneven due to 82 GB, but it will list distribution). The scheduler will try to keep processes on the same node as their memory. If the storage server is running something like a database or a heavy application in addition to file serving, you might consider CPU affinity or taskset to bind processes to certain CPUs if needed for performance consistency. However, for pure file serving, the defaults usually work fine.
- **Device Drivers for Add-ons:** If adding a 10GbE NIC, ensure the driver is available: e.g., Intel X520 uses `ixgbe` driver (in-kernel), Mellanox ConnectX-3 uses `mlx4` driver, etc. These are standard in enterprise Linux kernels. Essentially, the Linux kernel (especially any modern LTS or distribution kernel from 2015+ which one would likely run on this 2025 setup) already contains support for all hardware that was out by 2014 (when this board was current) and much beyond.
- **UEFI vs Legacy Boot:** The board has a 16 MB AMI BIOS that likely supports both Legacy BIOS and (limited) UEFI booting ⁶⁹. For Linux, one can boot in either mode. If using very large boot drives (>2 TB), UEFI boot might be needed to handle GPT boot, but usually one would have a small boot volume. The BIOS supports PXE boot on the i350 NIC, which could be used for network booting Linux installers etc.
- **Linux Distribution Compatibility:** The X9QRi-F+ is an "enterprise server" platform, so it is certified or used with OSes like RHEL, CentOS, SLES, Ubuntu Server, etc. The Supermicro site's OS

compatibility likely lists various Linux versions. By 2025, even the latest Ubuntu or RHEL should install cleanly on it. There is no proprietary technology that lacks drivers (no weird RAID or NIC that isn't supported). Even IPMI can be integrated (OpenIPMI services, or one can ignore and manage IPMI externally).

- **System Configuration for Storage Role:** Under Linux, after ensuring hardware works, one would configure:
 - Disk arrays with either **mdadm** (for classic RAID) or ZFS (if using ZFS on Linux via `zfsutils`). Both will use the CPU and memory resources effectively. For `mdadm` RAID, setting the stripe cache size (for RAID5/6) and using bitmap for rebuild might be considered. For ZFS, one would tune ARC size (maybe allocate a good portion of 82 GB to ARC for caching).
 - Filesystem choices: XFS or EXT4 are typical for `mdraid` arrays. XFS is often preferred for large arrays (has good parallelism and reliability). EXT4 is fine too. If using ZFS, it's both the RAID and filesystem, providing end-to-end checksumming which pairs nicely with ECC memory to protect against bit flips.
 - **NUMA tuning:** for example, if using XFS/MDRAID and heavy network usage, one might consider IRQ affinity: bind the HBA interrupt to the CPU connected to it, and the NIC interrupt to perhaps a CPU that's connected or at least isolate them. Linux allows setting IRQ affinity masks.
 - **Network config:** possibly use `bond0` combining the two 1GbE for management or separate networks, but more likely, disable one or use it as a failover for the primary NIC if 10GbE is added.
- **Power management settings:** In BIOS, one might choose a performance profile to avoid C-states if low latency is desired, but that increases power. Or enable C-states for power savings if the server is not always busy. Linux can handle CPU C-states and frequency scaling (`acpi_cpufreq` driver, etc.). Given power concerns, enabling Intel P-state driver in Linux can throttle CPUs when not busy, which is beneficial in saving energy (with 4 CPUs, idle power can be cut significantly by dropping cores to 1.2GHz or parking cores).
- **Driver Stability:** All the relevant drivers (AHCI, `igb`, `ipmi_si`, etc.) are well-tested on this generation hardware. No special firmware needed except to ensure BIOS is updated for v2 CPUs (which we have assumed is done). IPMI firmware updates can be done if needed, but if it's working, not mandatory. Perhaps check if there are any known errata: sometimes Intel had C-state issues or so on early BIOS, but by now probably resolved.
- **Software Management Tools:** There are various open-source tools to manage a Linux storage server: for example, `smartmontools` to monitor disk health on all drives (works with SATA/SAS drives to read SMART data), `mdadm` for RAID monitoring (can set up email alerts for array issues), or ZFS's `zed` for event handling. The IPMI can also be leveraged: e.g., if a fan fails, IPMI traps can be caught; or one can poll IPMI sensors and feed into monitoring systems (like SNMP or `collectd`).

In summary, running Linux on this hardware is a natural fit – all components have **broad Linux support and stable drivers**. You won't need any exotic kernel modules or hacks. The server should run any modern Linux distro reliably. The system management tasks like remote console, power control via IPMI work independently of the OS, but Linux can integrate with them if desired. The biggest configuration steps will actually be tuning the storage software (RAID/ZFS) and network (maybe bonding or setting up NFS/SMB services). The hardware will be a transparent platform for Linux to do its job.

9. Performance and Benchmarks: Expected Throughput and Parallel Processing

When evaluating performance, we should consider two aspects of this system: **storage I/O throughput** (especially in RAID configurations) and **compute capability for parallel processing** (which could involve RAID parity calculations, compression, or serving concurrent clients). While specific benchmark results for exactly this configuration might not be readily available, we can extrapolate from similar setups and known hardware limits:

Disk I/O Throughput: With a mix of SSDs and HDDs in RAID, the throughput will heavily depend on how the disks are organized: - If using a **RAID-10 of HDDs** (say 4 or 6 drives), one might expect sequential read/write in the range of a few hundred MB/s (perhaps 300–600 MB/s). HDDs in RAID-10 will get some benefit of striping (maybe $\sim N/2$ times the performance of one disk). 6 fast SATA HDDs could approach ~ 1 GB/s read (if each is 180 MB/s, striped across 3 pairs, ~ 540 MB/s, with some head movement, etc.). Latency for small random I/O would still be limited by the ~ 10 ms seek of drives, but multiple drives provide better IOPS scaling. 6 drives could yield maybe ~ 600 IOPS instead of 100, if they all seek in parallel (roughly linear with count for random). - If using **SSDs in RAID-0 or RAID-5**, throughput can be very high. A single SATA SSD can do 500–550 MB/s sequential. In a RAID-0 of 4 SSDs, one could see ~ 2 GB/s. We have the earlier example: **8 SATA SSDs on an LSI 9207 HBA reached ~ 4.1 GB/s** sequential read ⁴⁸ ⁴⁹. That's near the max of the PCIe x8 link. With fewer SSDs, say 4, on a single 6G HBA, we might see ~ 2 – 2.2 GB/s (which might saturate a PCIe 2.0 x8 or stay within PCIe 3.0 x4). - If using a **RAID-5/6 with HDDs**, write performance can be the limiting factor (due to parity overhead and random write penalty). However, this system's CPU can handle parity calc well, so it's more about disk speed. For example, a 5-drive RAID-5 of 7200RPM HDDs might give ~ 500 MB/s reads (since 4 data stripes read in parallel), but writes may be lower due to read-modify-write cycles (maybe half of that, unless doing large sequential writes which can use stripe-aligned full writes). - **RAID-5/6 with SSDs:** The CPU can calculate parity at several GB/s (especially with AVX2, though these are AVX1 era, still fine). The limitation would likely be the SSDs and controller. 4 SSDs in RAID-5 might have writes around the speed of 3 SSDs (since parity overhead) – e.g. if each is 500 MB/s, maybe 1500 MB/s writes, and reads around 2000 MB/s. These are ballpark. Such a setup would be entirely bottlenecked by the 1GbE network if only using onboard NIC, which is why a 10GbE is needed to see these benefits externally.

Networking Throughput: On dual GbE, the max combined output is ~ 2 Gbps (~ 250 MB/s). This is easily achieved even by a single modern HDD or a couple of them in RAID. So a single large file transfer could saturate one 1Gb link at ~ 110 MB/s. With multiple transfers, both ports can be used either separately or bonded. But either way, if the storage array can push GB/s, the network is the choke if left at 1Gb. With 10GbE, you open up to ~ 1.1 GB/s per link (for reference, a single SATA SSD's sequential speed is ~ 0.5 GB/s, so even one or two SSDs can saturate 10GbE). For truly maxing out a multi-SSD array, multiple 10GbE or a 40GbE would be needed. The system can support that as discussed.

Memory and Cache Effects: With 82 GB RAM, a lot of reads might come from cache if the same data is accessed repeatedly. Linux will buffer file reads in memory (as page cache). So if, for instance, a set of files < 82 GB is frequently read, after initial load they'll be served from RAM at basically memory speed. That could be tens of GB/s internal throughput. But again, externally the network is the limiter. Internally, if there was a compute job reading data, it could get it from cache quickly. The memory bandwidth per CPU (~ 50 – 60 GB/s) means even with multiple threads doing I/O, the NUMA-local memory can supply data fast enough to not impede disk speeds.

Parallel Processing / CPU Performance: The four Xeon E5-4650 v2 (or 4657L v2) provide a large number of cores and threads (40 cores / 80 threads in total with 4x10-core + HT). This is beneficial if the storage server is performing not just simple file serving, but also things like compression, encryption, deduplication, or hosting VMs/containers.

- For example, if using **ZFS**, the server could enable compression (LZ4) which these CPUs can handle at hundreds of MB/s per core. With many cores, ZFS could potentially compress multi-gigabytes per second of throughput on the fly. Same for checksum calculations (ZFS Fletcher or SHA-256 checksums would be spread across cores).
- If doing **software RAID-6**, the dual parity calculations (like P and Q using Reed-Solomon) would normally be CPU-intensive, but the mdadm driver uses SIMD optimizations. The presence of AVX (and potentially the pclmulqdq instruction for carry-less multiply used in some RAID6 algorithms) means the CPU can accelerate these computations significantly. It's not unrealistic to see RAID6 parity generation well above 5 GB/s on a modern multi-core – certainly within what these chips can do collectively. So the CPU won't be a bottleneck for RAID parity in most scenarios; the drives will max out first.
- **IOPS and concurrency:** With 40 physical cores, the system can handle many threads doing I/O. For instance, it could serve dozens of simultaneous clients over NFS/CIFS without pegging the CPU. Each network interrupt and disk interrupt can be spread over cores (by IRQ affinity or Linux's multiqueue block layer). The result is that even under heavy multi-user load, the CPU usage might remain moderate on each core, using maybe 10-20 cores fully and leaving others for other tasks or spikes. This is a big advantage over a single or dual-socket system when dealing with large-scale workloads.

Benchmarks Referenced: We have some references to generational improvements: - The E5-4600 v2 family offered *"up to 40% higher performance on average over previous generation"* (v1) ⁷⁰. This just indicates these CPUs are quite capable in their generation. But relative to newer CPUs (e.g. a single modern Xeon Scalable), the per-core is lower. Still, many workloads (like serving files) scale with number of cores rather than single-thread performance, so 40 slower cores can outperform 16 faster cores if the load is parallelizable (which it usually is in a server environment with many requests). - Memory performance got a 16–20% uplift in v2 as well ⁷¹, which helps in memory-heavy tasks (like deduplication or caching).

For **storage throughput** benchmarks on similar systems: - A Dell PowerEdge R820 (which is a 4-socket E5-4600 v2 server) with a RAID card can push a lot of data. Although we don't have a direct number, we could reference that such enterprise 4P servers are used to consolidate workloads – in our case, expect that any single RAID array's sequential throughput will be limited by the storage medium, not the server. E.g., if you have a 8-disk RAID-10 of 15k SAS drives, expect on the order of ~1 GB/s sequential reads. If it's 8 SSDs, expect ~4 GB/s as mentioned.

Parallel File Serving / Network: If multiple clients on a 10GbE or multiple 1GbEs are reading different things, the server can handle it. For instance, one could simulate with a tool like fio or Iometer multiple threads reading from a large file dataset on the RAID – the disk array will deliver IOPS and throughput depending on RAID level. The CPU can handle context switching and data movement easily in parallel. The limiting factors might be: - Disk seek contention if many random reads on HDDs (SSDs alleviate this). - Network saturation if many clients sum up beyond 1GbE. - Possibly lock contention at the filesystem layer if all clients hit the same file (less likely with multiple files).

Benchmark example: Consider a scenario: 10 clients each copy a large file from the server concurrently. If the server has, say, a RAID-6 of 8 drives (mix of SSD/HDD), it might have ~1–2 GB/s of total read throughput available. If network is 10GbE (~1.1 GB/s), it can only send out that much at once. So if 10 clients, each might get ~100 MB/s in that scenario. The CPU usage might be, say, 10–20% overall (with some threads handling network, some handling disk).

Now, if we had dual 10GbE (20 Gbps) and aggregated or serving to multiple networks, the server could theoretically push ~2.2 GB/s out. That would then likely stress the storage more. But the platform has the resources to handle it – the PCIe can support a dual-port 10GbE and an HBA simultaneously at full tilt.

Local Processing: If the storage hub is not just a NAS but also runs local compute tasks (like maybe data indexing, virus scanning, or some analytics on the stored data), those tasks can use the 40 cores. The NUMA nature means memory-bound tasks might need to be NUMA-aware to get best performance. But for embarrassingly parallel tasks (like scanning different files), one could pin threads to each CPU node and get nearly linear scaling.

Potential Bottlenecks: We should identify what could limit performance in certain cases: - **Network Bottleneck:** As noted, if only using 1GbE, the network is the throughput bottleneck for large transfers. Even with 10GbE, if the storage is extremely fast (like multiple SSDs), a single 10GbE link may become the new bottleneck at ~1.1 GB/s. For maximum, multiple 10Gb links or a 40Gb link would be needed to fully not bottleneck a very high-end all-SSD array. - **Storage Bottleneck:** If mostly HDDs are used, random I/O performance will be the bottleneck (the mechanical latency). The CPUs can queue a lot of I/O, but disks only do ~200 IOPS each. The large RAM can mitigate some of this via caching (reads) and write-back (if using hardware RAID cache or ZFS with ZIL etc. maybe using SSD for write log). - **QPI Bottleneck:** In extreme cases where, say, all NIC traffic goes into CPU4, but data is on memory attached to CPU1 (maybe because a process on CPU1 is handling it, or mis-pinning), the QPI links (8 GT/s each) could theoretically saturate. But 8 GT/s QPI is about 32 Gbps per direction raw, likely ~25 Gbps effective. That's ~3.1 GB/s cross-socket. So if more than 3 GB/s of traffic needs to go from one CPU's memory to another's NIC, you'd max the QPI. But that scenario can be avoided with good NUMA strategy (e.g., run the file server daemon on the CPU that has the NIC and/or place its buffers on that node). - **CPU Bottleneck:** If doing something heavy like software encryption on all traffic (e.g., IPsec or TLS for every file transfer, or using full-disk encryption with dm-crypt on a multi-gigabit stream), then CPU could become a bottleneck. AES-NI helps a lot though – these CPUs have AES-NI, which can encrypt at multi-gigabit speeds per core (roughly, AES-128-GCM can do a few hundred MB/s per core on Ivy Bridge, so with 40 cores, even 10+ GB/s aggregate encryption is possible). So likely fine, but a noteworthy point if someone turned on encryption and compression everywhere, then maybe CPU usage would climb.

Benchmark references: If we were to cite an official or semi-official benchmark: - The Intel brief might have shown SPECint or STREAM memory numbers, but not directly storage. - Possibly, a tool like **IOmeter** or **FIO** run on a similar machine would produce numbers: e.g., *“eight Crucial SSDs... 4.1 GB/s”* we already cited ⁴⁸. We could also mention that this test also yielded very high IOPS (eight SSDs could do on the order of 400k IOPS 4K random reads, limited by HBA and latencies). - **Level1Techs forum** had someone asking about E5-4600 v2 benchmarks ⁷², presumably for compute tasks, but not specific results for storage.

We can also allude to real-world tasks: - For instance, **ZFS build with similar hardware:** some user experiences show saturating 10GbE with a mirror of SSDs or a RAID-Z of many HDDs is easily done by these CPUs, often the disks or network limit out before CPU. - **Parallel compilation or compression:** If one

repurposed this as a build server or something, 40 cores would compile massive projects quickly. But for storage focus, if you compress data (like tar.gz backups), having many cores means you could run, say, 40 compression threads in parallel to speed up archiving of many files.

To sum up, **expected performance**: - *Throughput*: With proper RAID cards and 10GbE, expect on the order of **1–4 GB/s** sequential throughput internally from disk, and up to the network limit externally (e.g. ~1 GB/s on 10GbE). In a fully SSD scenario, the array could exceed what a single 10Gb link can carry ⁴⁸, so multiple links would be needed to see the full potential. - *IOPS*: With HDDs, maybe a few thousand IOPS (if say 10 drives at ~150 IOPS each in random workload). With SSDs, tens of thousands to hundreds of thousands of IOPS are possible (SATA SSDs do ~80k IOPS each 4K random, 8 of them could approach ~640k IOPS at the controller, though the HBA and CPU overhead might cap lower). The CPUs can handle interrupts for those IOPS (it might distribute across cores). - *Parallel CPU tasks*: The system could handle **80 simultaneous software threads**. If those threads are doing relatively light work (like serving files, waiting on I/O), you can pack a lot of them. If they're heavy (like compressing), you have to consider each core has performance roughly like an Ivy Bridge at 2.4 GHz (somewhere between a modern Atom and a modern Xeon core in single thread). But multiply by 40 and you have something roughly equivalent to, say, 2 modern high-end CPUs (for instance, a single Xeon Gold 5218 has 16 cores of newer arch ~2.3 GHz, two of those ~32 cores, so 4 Ivy Bridge = maybe ~1.5 × performance of 2 modern, depending on scaling and per-core differences).

Given the target use (storage hub), typical **benchmarks** we might reference or suggest include: - **fio (flexible I/O tester)**: e.g. measuring throughput and IOPS for various block sizes. We might say: try a scenario with

```
fio --filename=<raid_device> --direct=1 --rw=read --bs=1M --iodepth=32 --numjobs=8
```

to simulate multi-threaded large reads – one might observe the array hitting ~2–3 GB/s (if SSD-based) or a lower number if HDD-based. Similarly random 4K reads/writes with multiple jobs to measure IOPS. - **bonnie++** or **IOzone**: these are filesystem-level benchmarks that could be used; bonnie++ would show sequential and random performance as well as create/delete ops. - **netperf** or **iperf**: to test network throughput (especially if 10GbE is added, one can measure ~9.4 Gbps on iperf easily if tuning, or if using multiple streams, saturate). - **SpecSFS** (SPEC storage benchmark) or even a simple Samba throughput test (like copying a file from a Windows client over SMB with multi-channel if using SMB3 multi-channel on dual NICs). - **Phoronix test suite** might have some relevant disk server benchmarks, but those might be overkill.

Since the question asks for *suggested or referenced benchmarks*, we should present a couple of credible figures: One we have is the 8 SSD 4.1 GB/s figure ⁴⁸ ⁴⁹. Another could be something like: Perhaps referencing that a single i350 port does ~940 Mbps in iperf (which is known, but could cite a known stat). But it's fine to just state that saturating Gigabit is trivial.

Another reference: According to Intel, the memory bandwidth of each CPU ~59.7 GB/s at 1866 ²⁷, which implies the memory subsystem is not a limiting factor for <10 GB/s of I/O flows, and our network won't get near that anyway.

Parallel processing wise, one metric is SPECint_rate or SPECfp_rate for multi-CPU which would be high – but storage context might not need that explicitly.

To be safe with citations: - We have [38] for the storage throughput. - We could cite [20] or [22] or [37] for improvements but maybe not necessary. - Possibly [22†L1–L4] which shows the top CPU's specs including “8.0 GT/s 1866” which indirectly confirms memory and QPI bandwidth available.

We already have [22†L1-L4] cited and [38], which should suffice.

In summary, this platform, when properly configured, can deliver excellent storage performance: - It can fully saturate multi-gigabit links with data (for example, pushing ~1000 MB/s over 10GbE, given a fast enough disk array). - It can handle simultaneous tasks (like computing parity during a RAID rebuild while still serving clients) due to abundant cores. - For benchmark numbers: one could expect **~4 GB/s sequential throughput** from an 8-SSD RAID-0 array locally ⁴⁸ ⁴⁹ ; around **1–2 GB/s** from a more modest array or with parity overhead; and easily **>100k IOPS** in random reads if using SSDs. Over the network, with only 1GbE, limited to ~110 MB/s per link; with 10GbE, up to ~1.1 GB/s, which such an array can saturate with headroom.

10. Use-Case Analysis: Quad-Socket Storage Server vs Newer Platforms

In this section, we evaluate the X9QRi-F+ quad-Xeon system as a dedicated storage server, considering its advantages and drawbacks in the context of modern alternatives.

Strengths of the X9QRi-F+ Platform as a Storage Hub:

- **Massive Parallelism:** With up to 40 cores / 80 threads, this server can handle a highly concurrent workload. For a storage server, this means it can serve many clients simultaneously, handle multiple RAID rebuild threads, encryption tasks, or compression tasks in parallel. Newer single- or dual-socket platforms might have high core counts too (e.g., AMD EPYC can have 64 cores on one chip), but in the era this was built, 4-socket was the way to get 40+ cores. Even today, 40 cores is substantial for I/O handling. This is advantageous if the storage server doubles as a compute server (running VMs or heavy file indexing, etc.).
- **Large Memory Capacity:** Supporting up to 1 TB RAM (and practically using tens or hundreds of GB affordably), it allows huge disk caches and buffers. Our 82 GB is relatively modest compared to the 1 TB max, but still, 82 GB is plenty for caching a lot of filesystem data. Many modern NAS appliances skimp on RAM (perhaps 16–32 GB in smaller units), whereas here we can cache aggressively, reducing disk reads and improving performance for repeated access patterns. Also, the ECC RAM ensures reliability for cached data.
- **High I/O Expandability:** The four PCIe 3.0 slots give it flexibility to integrate new technologies. We can add 10GbE or even 40GbE NICs to upgrade network bandwidth – something mandatory to fully utilize a fast disk subsystem. We can also add the latest storage controllers or even NVMe drives via adapters. While the board itself is from 2013 era, it can accommodate, for example, a 2019 NVMe SSD in a PCIe slot (though at PCIe 3.0 speeds). This extendability means the system can be upgraded to keep pace with certain advancements (to a point).
- **Proven Reliability and Support:** Enterprise boards like this have validated designs and often long-term support (BIOS updates, etc., though by 2025 it's EOL, but during its prime it had support). The use of server-grade components (Intel NIC, IPMI, ECC memory) and form factor means it's built for 24/7 operation. For a storage server where uptime and data integrity are paramount, this is important. Many “newer” platforms like modern consumer boards with lots of SATA might not have IPMI or ECC, which are big pluses for reliability and remote management.
- **Cost Efficiency (used market):** Although not an “in-technical-spec” advantage, by 2025 this platform can be obtained relatively inexpensively on the secondary market, while still delivering very solid performance. The ServeTheHome forum notes that 4× E5-4650 v2 CPUs can be found for relatively low cost now ⁷³ . This means you can build a powerful storage server for a fraction of the cost of a brand-new enterprise server. For a certain audience (like homelabs or budget-conscious enterprises), this is a practical advantage.

Advantages over Newer Platforms: It might seem counter-intuitive that an older platform has advantages over newer ones, but there are a few scenarios: - **Memory and I/O Bandwidth vs cost:** Many newer single-socket solutions (like Xeon Scalable Silver/Gold or AMD EPYC) offer huge performance, but the platforms and memory (DDR4/DDR5) are expensive. The X9QRI-F+ can use cheaper DDR3 ECC modules readily available used ⁷⁴. If one needs a very large memory pool (hundreds of GB) for a storage server (for caching or in-memory datasets), populating this board might be cheaper than buying high-capacity DDR4/DDR5 for a new system. And its aggregate memory bandwidth (4 CPUs × 4 channels each) is still quite high (theoretically ~200 GB/s) – a single newer CPU might have 8 channels at faster DDR4, but e.g. 8×25.6 GB/s = ~204 GB/s, so not far off. - **PCIe Slots Count:** Some modern single-socket boards (e.g., ATX boards) might have fewer PCIe slots (maybe 3 or 4 total, often with lane bifurcation for M.2). The X9QRI-F+ gives 4 full slots without needing to share lanes with M.2 devices etc. A modern dual-socket might equal or exceed that, but single-socket mainstream could be limited. For example, if comparing to a Xeon D appliance or a smaller server board, those often have limited expansion. - **NUMA segregation for performance isolation:** With 4 CPUs, one could dedicate certain tasks to specific CPUs. For example, you could run a database server on CPU3 with its local memory, and the file server on CPU1, somewhat isolating their memory and CPU usage (they communicate via QPI if needed but can largely work separately). On a single-socket multi-core, all tasks share the same memory controller and caches, which is usually fine but in heavy multitasking, NUMA isolation can sometimes help performance tuning.

However, **there are notable disadvantages and bottlenecks** as well, especially compared to state-of-the-art: - **Power Efficiency:** Newer platforms (like 2nd Gen Xeon Scalable or AMD EPYC) deliver much better performance per watt. A single EPYC 7302 (16-core) might equal or beat two of these old 10-core Xeons in many workloads while consuming far less power. The X9QRI-F+ with four CPUs will draw significantly more power at idle and load. This affects operating cost (electricity, cooling requirements) and makes the system less “green.” If this is an always-on storage server, over years the power bill difference might justify a newer more efficient system. - **Heat and Noise:** As discussed, this system requires strong cooling. Modern CPUs are also hot, but you might have half as many in a simpler configuration. A new system with a single 32-core CPU might be easier to cool (and quieter) than four 10-core ones. If the storage server is in a non-datacenter environment, noise could be an issue. The X9 will be loud when fans ramp up. - **Performance per Core:** Each Ivy Bridge core at 2.4 GHz is much slower in single-thread than a modern core at 3+ GHz with IPC improvements. Some storage tasks (like serving a single threaded request, or running a mostly single-threaded app) would be faster on a new CPU. If the workload cannot utilize many cores, the newer platform is preferable. For instance, metadata operations in a filesystem (like directory listings) often run single-threaded per request. A faster core would respond quicker. However, if many users do that, the old platform can handle concurrency, but any one operation might be slower than on a new CPU. - **Missing Modern Features:** Newer platforms have PCIe 4.0 or 5.0 (higher bandwidth for HBAs and NICs), NVMe U.2 or M.2 slots on board, built-in 10GbE on many server boards, and support for faster memory (DDR4/DDR5) which can help in some edge cases. The X9QRI-F+ is limited to PCIe 3.0 and no native NVMe. So high-end NVMe SSDs will be capped at ~3.3 GB/s on PCIe 3.0 x4. Also, things like NVMe-oF (NVMe over Fabrics) target mode likely run better on newer gear with dedicated NVMe slots and faster networking. If one wanted to, say, directly attach NVMe drives or use GPUs (for some reason in a storage/AI combo server), a new platform with PCIe4/5 would greatly outperform. - **Support & Lifecycle:** By 2025, this platform is long out of official support (EOL). Firmware updates (BIOS, BMC) are no longer released. Newer OS features might not be validated on it. In contrast, a new platform would have ongoing updates and security fixes (e.g., new mitigations for side-channel attacks might not be back-ported to older microcode here). That might be a consideration for enterprise use (security). - **Density:** A modern single-socket 1U could potentially replace a larger 4U older system in terms of performance, while consuming less space and power. If datacenter rack

space is a factor, consolidating might be beneficial. However, if you already have the chassis and not constrained, this is not a big issue.

Potential Bottlenecks Recap: - **Network Bottleneck:** On our platform specifically, if it's not upgraded, the dual 1GbE is a bottleneck. But we assume we can upgrade via PCIe (which is an extra cost/complexity compared to newer boards that might have 10G built-in). - **Single-thread bottlenecks:** As noted, any single client's performance (latency or single-stream bandwidth) might be limited by one core's performance or one link's capacity. Newer tech could have higher per-thread performance and features like RDMA (Remote Direct Memory Access with modern NICs, though one can add RDMA NIC here too). - **Cross-CPU bottlenecks (NUMA):** If not carefully managed, memory access across QPI could become a bottleneck for some workflows. For example, if a RAID card on CPU2 is frequently accessed by a process running on CPU1, the QPI latency will add. In a newer single-socket, no NUMA, so simpler. However, OS can mitigate with NUMA pinning or one can ensure the major services are local to their I/O. This is just a complexity factor that doesn't exist on single-socket systems.

Ideal Use Case for X9QRI-F+: It shines in a scenario where you need a large amount of resources thrown at a problem without high incremental cost: - As a dedicated storage server, especially if you combine roles (maybe it's also doing processing on the data). For example, a Hadoop or distributed storage node with compute, where having 4 CPUs can do storage management and run data tasks. - As a virtualization host with a lot of local storage (the cores can host many VMs/containers and the drives give large storage pool). Though storage in virtualization nowadays often uses centralized storage, for smaller setups this could be an all-in-one. - If you specifically need lots of PCIe devices (maybe multiple HBAs connecting to lots of JBOD enclosures, plus multiple NICs), the quad-socket can handle that addressing and interrupts across CPUs nicely.

Compared to a New 2025 platform (for example, a single-socket AMD EPYC 7552 with 48 cores, PCIe4, etc.): - The new one would have higher memory bandwidth (8-ch DDR4 or DDR5), much higher per-core speed, more PCIe lanes (128 lanes PCIe4), and integrated features, all in a simpler package. It would likely outperform the old one in almost every metric while using less power, except possibly raw memory capacity if the new one didn't have as many DIMM slots (though many new boards support 16 DIMMs per socket, which with 64GB modules is already 1TB, so moot). - The trade-off is cost: that EPYC board + CPU might cost a lot more than repurposing an existing X9 server. - So one could say: the X9QRI-F+ is "*advantages over newer platforms*" mainly in **cost for performance (on second-hand market)** and **the ability to reuse existing DDR3 memory and older enterprise parts**. It also runs legacy OS easily (if someone needed to run an older OS for compatibility, though for a new deployment one would use a current OS).

Potential Bottlenecks Already Identified: network, storage interface, single-thread, power/cooling. We should ensure we articulate them: - Network: limited onboard to 1GbE (need upgrade). - DMI: limits onboard SATA aggregate (~2GB/s). - QPI: remote memory/disk access overhead, as described. - Possibly the generation's lack of certain acceleration (no NVMe, no PCIe4, etc., which we've covered). - Disk count: only 6 onboard without HBA (again solved by expansions).

Long-term considerations: Another soft point: as technology moves, newer NVMe drives or such might saturate PCIe3 quickly. If down the road one wanted to attach NVMe U.2 SSDs (via an HBA like Broadcom 9400 series or similar), each NVMe can do 3-4 GB/s, and a PCIe3 x8 HBA might handle maybe 2 of those at full speed. Newer boards with PCIe4 could handle twice as much per slot. So for extremely high-end all-

NVMe storage, this older board would be the limiting factor eventually. But for a mix of SATA/SAS drives, PCIe3 is fine.

Conclusion on use-case: As a dedicated storage server, the X9QRI-F+ is very capable – it offers enterprise reliability (ECC, IPMI), high concurrency, and flexibility to attach lots of drives and networks. It can absolutely function as a central storage hub for a lab or a data center tier (like a backup server or secondary storage node). Its advantages are more prominent if one already has the hardware or can get it cheaply, and if the power usage is not a major concern. Against newer platforms, it's largely outclassed in efficiency and simplicity (newer one might do the same job with fewer CPUs), but not necessarily in raw I/O performance if both are equipped similarly. With proper tuning, a quad E5-v2 system can still saturate modern storage devices and network links to deliver high throughput.

Bottlenecks to monitor include: - **Network** – likely the first limiting factor; solve by adding 10GbE. - **Disk I/O** – ensure use of HBAs to avoid the SATA port bottleneck. - **NUMA alignment** – avoid cross-node penalties by configuring services appropriately. - **Power/Cooling** – make sure the environment supports it, otherwise thermal throttling or component strain could become an issue (which is a “bottleneck” in a sense).

Finally, one might consider the maintenance: with more components (4 CPUs, 32 DIMMs), there's a larger surface for potential failure (like more DIMMs that could go bad, etc.) compared to a simpler system. But that's mitigated by good hardware (ECC catches memory errors, etc.) and possibly by having spares if needed.

In summary, the X9QRI-F+ is “overkill” in some ways for a storage server but that overkill translates to headroom and robustness. It's an older tank of a server board that, when outfitted correctly, can still deliver enterprise-level storage performance. However, the user should be aware of its hunger for power and cooling, and the need to augment it with modern interconnects (like 10GbE) to truly leverage its capabilities in a present-day network.

Citations already given in prior sections cover most points, but we may want to cite a line or two about the memory or lane count: We cited that it supports up to 1TB memory ³¹, which is a stand-in for large capacity advantage. Also the STH forum on value: - We saw serveTheHome forum “best value E5-4600v2 CPU” ⁷³ which indicates they were cheap (\$60 each for 4650 v2 in 2019, though prices went up by the time of that post – but still, relative to original \$3600 price ⁷⁵, that's pennies on the dollar). But we've loaded enough citations. Perhaps we could slip in the one about used prices as anecdotal: It says: “I got four E5-4650 V2 for \$60 each... E5-4657L V2 was around \$95-105 at that time” ⁷³. This highlights the cost aspect. Possibly worth citing to emphasize the cost advantage nowadays.

And maybe cite memory support [19†L9-L17] again or [12†L201-L209] for 1TB and 32 slots if needed (we did earlier though).

We might not need many new cites here, just recapitulate and sum up.

Let's finalize Section 10 with that perspective.

1 3 5 8 9 11 24 26 31 34 35 39 55 57 63 64 65 66 67 68 69 **server-shop.ua**

https://server-shop.ua/assets/images/resources/8409/products-supermicro-motherboard-x9qri-fspecification.pdf?srsId=AfmBOoeHnE8m0272xZaVrDfAU809HuAWgNmZZVKZnAzhDRWRUz2C_Pd

2 14 19 20 21 22 23 27 28 29 37 54 75 **Intel® Xeon® Processor E5-4650 v2**

<https://www.intel.com/content/www/us/en/products/sku/75289/intel-xeon-processor-e54650-v2-25m-cache-2-40-ghz/specifications.html>

4 6 **CPU & Memory options for X9QRi-F+ | ServeTheHome Forums**

<https://forums.servethehome.com/index.php?threads/cpu-memory-options-for-x9qri-f.25318/>

7 70 **The Power of Compute in Dense 4-Socket Servers**

<https://www.intel.com/content/dam/www/public/us/en/documents/product-briefs/xeon-e5-4600-v2-brief.pdf>

10 30 38 40 52 56 **Configure a PC with Supermicro X9QRi-F+**

https://www.pugetsystems.com/parts/Motherboard/Supermicro-X9QRi-F-9885/?srsId=AfmBOoqFa49z4SQw6P9IkfK14TZd0extLqE2Y_oIBeRlJJA-7mAXd-D

12 13 15 16 18 25 **Intel® Xeon® Processor E5-4657L v2**

<https://www.intel.com/content/www/us/en/products/sku/75290/intel-xeon-processor-e54657l-v2-30m-cache-2-40-ghz/specifications.html>

17 **HPE Intel Xeon E5-4600 v2 E5-4650 727576-B21 | Tech-America**

<https://www.tech-america.com/item/hpe-intel-xeon-e5-4600-v2-e5-4650-v2-deca-core-10-core-2-40-ghz-processor-upgrade/727576-b21?srsId=AfmBOopufAoL29mDOfd0ckHfb9kBjuDu8nu-cjM1SR5HXLrhVUL4vsYv>

32 **Amazon.com: Supermicro X9QRi-F+ Motherboard : Electronics**

<https://www.amazon.com/Supermicro-MBD-X9QRi-F-B-X9QRi-F-Motherboard/dp/B00B2W81LK>

33 **Archive Archived Motherboards (End-of-Life) - Supermicro**

<https://www.supermicro.com/products/motherboard/archive/>

36 **Memory population for Supermicro X9QRi-F+**

<https://forums.servethehome.com/index.php?threads/memory-population-for-supermicro-x9qri-f.37184/>

41 **General references and recommendations for home server rebuild | ServeTheHome Forums**

<https://forums.servethehome.com/index.php?threads/general-references-and-recommendations-for-home-server-rebuild.45710/page-2>

42 43 44 51 **LSI HBA card recommendations? (UK based) : r/unRAID**

https://www.reddit.com/r/unRAID/comments/v5lyxo/lsi_hba_card_recommendations_uk_based/

45 46 47 58 59 **Top Picks for FreeNAS HBAs (Host Bus Adapters)**

<https://www.servethehome.com/buyers-guides/top-hardware-components-for-truenas-freenas-nas-servers/top-picks-truenas-freenas-hbas/>

48 49 50 **LSI SAS 9207-8i PCIe 3.0 HBA Overview - Eight Crucial M4 SSDs Pushed to 4.1GB/s Performance | The SSD Review**

<https://www.thessdreview.com/our-reviews/sata-3/lsi-sas-9207-8i-pcie-3-0-host-bus-adapter-quick-preview/>

53 62 **Tech 4 Drive | X9QRi-F+ SuperMicro Quad Socket LGA 2011 Intel C602 Chipset Xeon E5-4600 v2/ E5-4600 Series Processors Support DDR3 32x DIMM 2x SATA2 3.0Gb/s Proprietary Server Motherboard (Refurbished)**

https://tech4drive.com/product/x9qri-f?srsId=AfmBOopdCiemriYa3GU_rphQalGGsEJQz3Nv84Sq76sr90d1sU7oT31d

60 Intel® Network Adapter Driver for 82575/6, 82580, I350, and I210 ...

<https://www.intel.com/content/www/us/en/download/14098/intel-network-adapter-driver-for-82575-6-82580-i350-and-i210-211-based-gigabit-network-connections-for-linux.html>

61 Supermicro MBD-X9QRi-F+ Quad Socket R(LGA 2011) 6x ... - eBiz Pc

https://www.ebizpc.com/Supermicro-MBD-X9QRi-F-Quad-Socket-R-LGA-2011-p/mbd-x9qri-fplus.htm?srsId=AfmBOoqguOZR4U1en6BurnBK0_-CEDqDD04dWKUj9aFzX46zzFOoY_gO

71 Intel Xeon E5-4600 v2 "Ivy Bridge" Processor Review - Microway

<https://www.microway.com/hpc-tech-tips/intel-xeon-e5-4600v2-ivy-bridge-processor-review/>

72 Benchmarks for Intel Xeon e5-4600 v2 CPUs - Level1Techs Forums

<https://forum.level1techs.com/t/benchmarks-for-intel-xeon-e5-4600-v2-cpus/192140>

73 what is the current best value E5-4600v2 CPU?

<https://forums.servethehome.com/index.php?threads/what-is-the-current-best-value-e5-4600v2-cpu.27714/>

74 Supermicro X9QRi-F+ Memory - Cloud Ninjas

<https://cloudninjas.com/collections/x9qri-fplus?page=4&srsId=AfmBOorRxx9UwG604iwoHILptC9ha2YYgDkqBQIz5Sib1uGB3zZSdV9B>